RailNetwork game

- project documentation -

designed and developed by Codrin Berinde
student, Technical University of Cluj-Napoca, Romania
semestrial assignment for Computer Architectures course (B35APO)
at the Faculty of Electrical Engineering
Czech Technical University in Prague
coordinating professor: Pavel Píša
31st May 2025

Contents

1 Project description **1**
1.1 Rail segments building mechanics 4

2 Hardware **6**

3 Software **6**
3.1 Data 8
3.1.1 The board 8
3.1.2 The buffer 10
3.1.3 The selected and *under construction* cells 10
3.2 Functions 11
3.2.1 main.c 11
3.2.2 play.c 11
3.2.3 events.c 14
3.2.4 rendering.c 14
3.2.5 generator.c 15

4 Future improvements **15**

1 Project description

The project is a game of 1 player. The board is a lattice of `BOARD_WIDTH` by `BOARD_HEIGHT` cells. The initial configuration is presented as a certain number `cities` of the cells in the lattice being occupied by so-called "cities". To each city is associated a numerical value (the "size" or "population") of the city. One such scenario is presented in Figure 1. The player begins with an initial number of points, `initial_points`.

Two cells are considered adjacent if they share a common edge (diagonal cells that share only a corner are not adjacent).

Between any two adjacent cells, the player is able to build a segment of railway. One segment of rail costs one point to build. Once a connection is made between some cities (i.e. a continuous path of railway segments exists between pairs of two cities), the player is rewarded

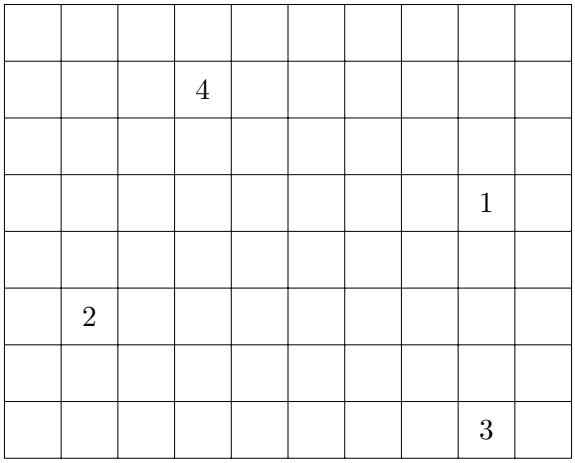


Figure 1: Initial conditions example

a number of points equal to the total size/sizes of the city/cities that through this connection are connected for the first time to other cities. When the first connection is made, obviously, there are two cities getting connected, therefore the player will receive a number of points equal to the sum of the sizes of the two cities.

Figure 2 depicts this scenario. The rails are represented in blue. The segments that have one end into a city have that end drawn as a ring around the number, but this is just a matter of representation; the way it works is the same as for all other segments. The player has chosen to connect the cities with sizes 2 and 4 on the shown route. Let's assume the game began with 7 initial points. Therefore, the player spent 6 points on the building of the tracks, because there are 6 segments of rail in between, and was rewarded $2+4=6$ as result of the connection.

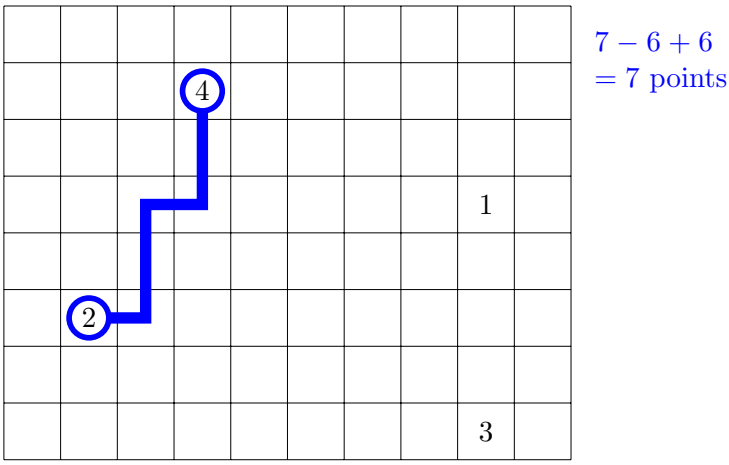


Figure 2: The first connection

Later, the player may either, for example, choose to connect the city 1 to the already-existing line, situation which is depicted in Figure 3 (lighter blue). However, in this case, the player would spend 5 points on building the line, and gain only 1, because the only city that through this connection gets access for the first time to other cities is the one with population

1. Therefore, now the player would have $7 - 5 + 1 = 3$ points. And the problem is that with 3 remaining points, city 3 cannot be connected to the network, because it is 4 segments away from the closest point in the network (which is city 1).

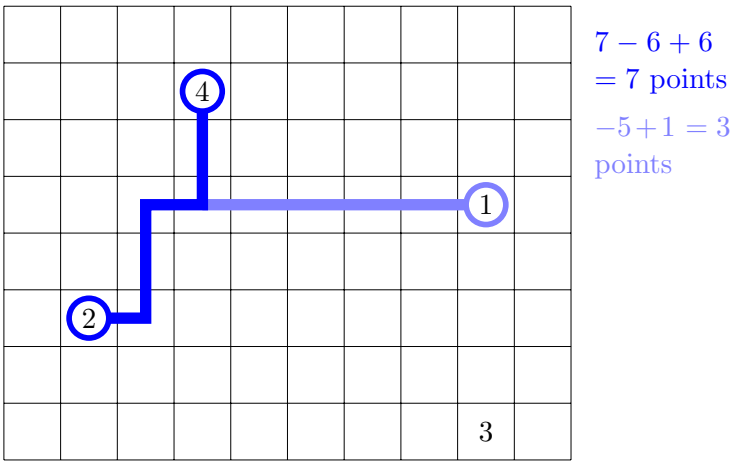


Figure 3: The second connection - bad move

Therefore, it is better that instead of this move, the player first connects the cities 1 and 3 between them, spends 4 points but also gains $1 + 3 = 4$ (from their population; they are connected for the first time), and now has a total of $7 - 4 + 4 = 7$ points (Figure 4 - light blue), which can be used (Figure 4 - very light blue) to later connect the two subnetworks into a single complete network. Because all the cities have previously been connected to some line, the player gains 0 points in this final connection, but it does not really matter anymore because all the cities have been connected.

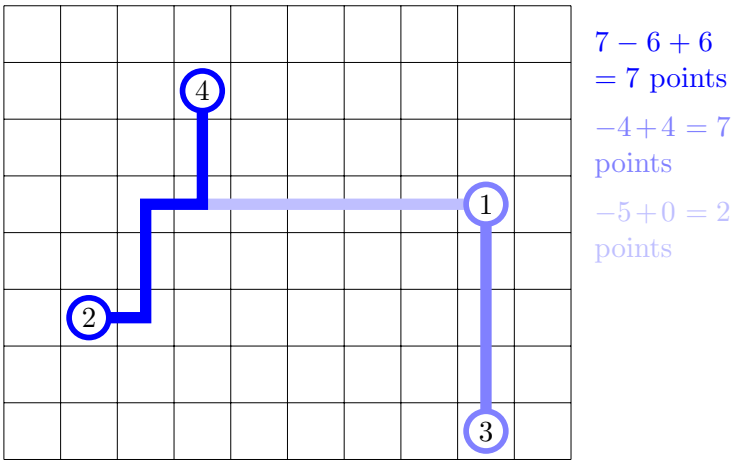


Figure 4: The second and third connections - smarter move

The goal of the game, as the reader might have figured out, is to connect all the cities into a single network.

Attention!

The points for building the rails are deduced at each step, after every segment, not at the end of a connection! Therefore, in another scenario, if the player with, let's say 3 points, attempts to connect a city with population 2 and a city with population 3 which are, let's say, 4 segments apart, this connection cannot be made, even though $3-4+2+3=4>0$, because once the line reaches 3 segments, the player has 0 remaining points, and there is still one more segment to build.

1.1 Rail segments building mechanics

Let us briefly describe how the player is supposed to take action on the board in order to build the rail segments in the desired positions. First, the most importat thing to mention is that on the board there is always an "active" cell, where the player currently is, which we will call the *selected* cell. This cell is drawn red on the screen. The player may move around with the selected cell by rotating the knobs: red knob to move up and down and green knob to move left and right. An example of selected cell is illustrated in Figure 5.

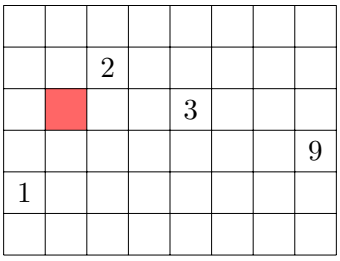


Figure 5: Selected cell example

Once the player decides to build a rail segment, they press on either the red or the green knob and that cell turns into *under construction* cell. This is marked by a green "target" - see Figure 6 a), and it represents the starting cell of the segment the player will build.

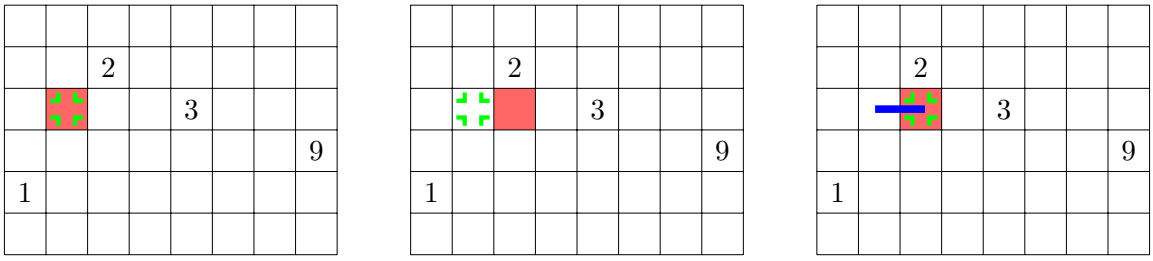


Figure 6: Building a segment

After the *under construction* cell is selected, the player moves to another current cell - the end of the segment the player wants to build - see Figure 6 b).

If the *under construction* cell and the current cell are adjacent and the player presses either the green or the red knob, then a segment will be built between the two. Simultaneously, the *under construction* cell is moved to the end of the newly-built segment - see Figure 6 c). It was made like this in order to ease the construction of a longer railway and not having to press on each cell two times.

If however the *under construction* cell and the current cell are not adjacent, but one of the red or green knobs was pressed, than no segment will be built, just the *under construction* cell will be moved to the new location (the current cell).

If the selected cell and the *under construction* cell coincide and one of the two knobs is pressed again, the *under construction* cell is deselected and there will be currently no cell under construction.

In order to build connections between cities, it is not mandatory to build all the segments in order, one after another. The player may choose to build the segments in any order they wish and unite them into a continuous line later. But the points are be gained only when an uninterrupted set of segments will exist from one city to the other.

2 Hardware

The project is implemented on the MicroZed APO Educational Kit developed at the Czech Technical University in Prague (ČVUT), featuring a Zynq™-7000 Dual ARM® Cortex™-A9 MPCore™ @ 866 MHz processor and multiple peripherals (see Figure 7), such as an LCD display (3), 3 knobs that can rotate and be pushed (4), each of them being provided with rotary encoders, a LED strip with 32 LEDs (5), two RGB LEDs (6). The board has Ethernet connection (1) and DC power supply (2).

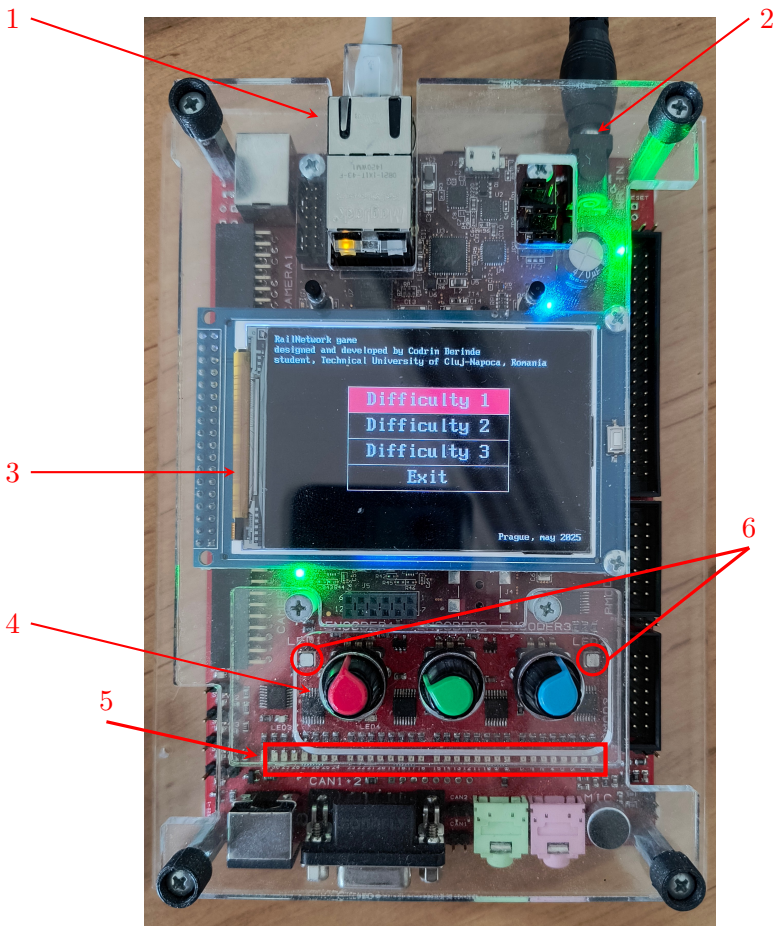


Figure 7: The MicroZed APO Educational Board

3 Software

The code for this project was written in the C language, and it is procedure-oriented, not object-oriented. The flow of information in the program is represented in Figure 8.

Every entity (vertical bar) that is shown in the flow diagram has all its belonging functions implemented in a separate `.c` file, with the same name as the entity, with the exception of the `game` entity, which is also implemented in the `play.c` file.

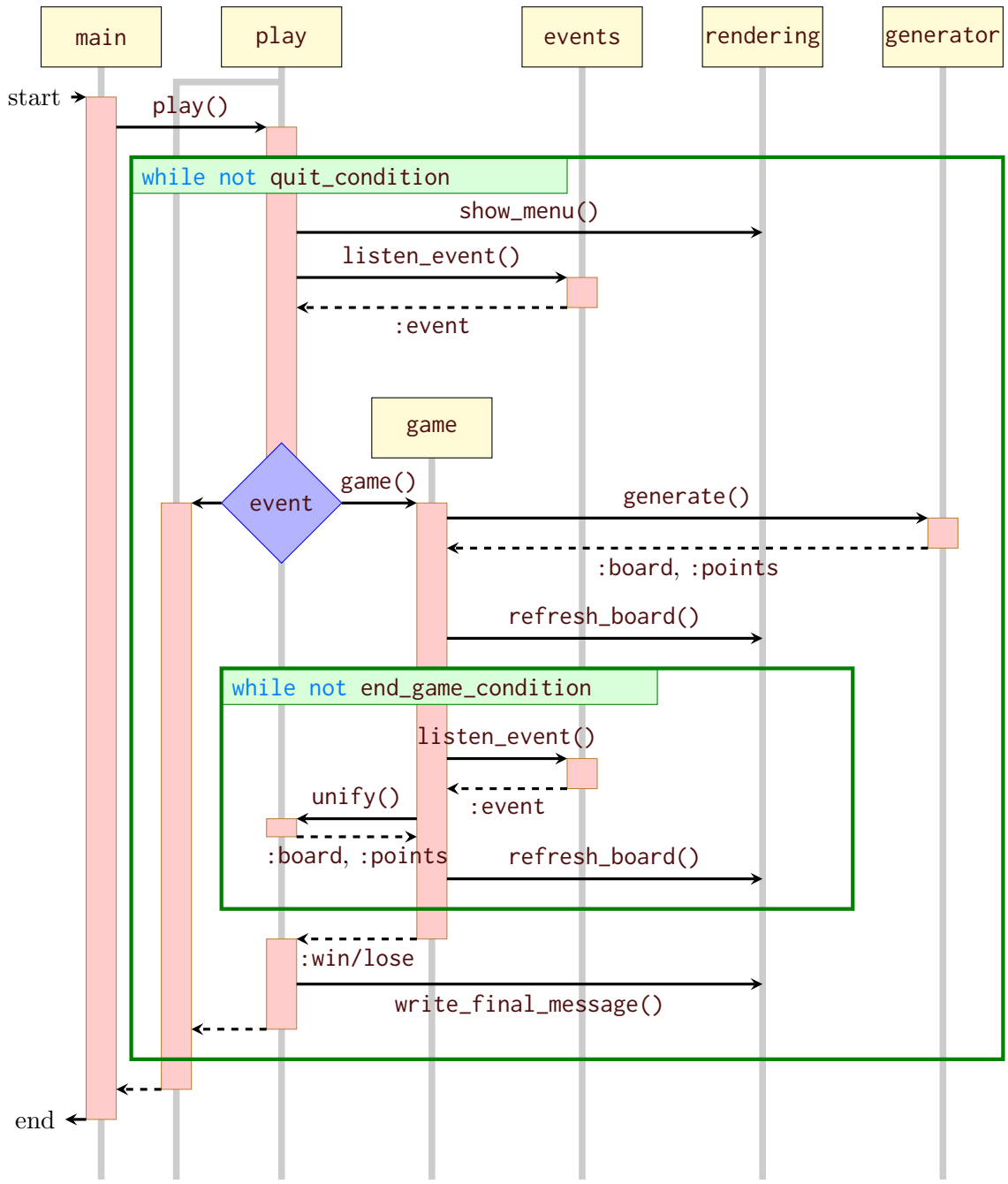


Figure 8: Flow of information into the program

To briefly explain this diagram, the `main` entity calls the `play()` function which belongs to the `play` entity. In the big loop, the main menu is displayed by the function `show_menu()` which belongs to `rendering`. Input events are listened to by the function `listen_event()` to capture any input data from the player (knob push or rotation). Based on the event that was caught at this point, the algorithm takes different paths. If only the selected menu option is

changed or there is no caught event (left branch), the loop runs again and the menu is displayed again. If the player chooses from the menu to start a game, the right branch is taken and the function `game()` is called. Upon its call, a new board is generated by the `generator` via the `generate()` function. The board and the initial number of points are returned. Then, the board is displayed on the screen by the `rendering` entity via the `refresh_board()` function. In the small loop, the algorithm listens to input events from the player, and based on them, modifies the board via the function `unify()` (adds segments / adds or subtracts points from the current number of points etc.), and of course the board is displayed again (according to the new modifications) by `refresh_board()`. This loop ends when the game is either won or lost, after which a final message ("You won!" / "You lost!") is displayed on the screen via `write_final_message()`. Then the menu is displayed again due to the big loop and the player may choose to start another game, or may choose to quit in which case the big loop terminates.

3.1 Data

In this section we will explore how the data for the game is stored: the current state of the board, the subnetworks etc.

3.1.1 The board

The board is stored in an instance of the structure defined in `play.h` as

```
typedef struct BOARD {
  uint8_t *data;
  int *parents;
} board_t;
```

`data` is a pointer to an array of `BOARD_WIDTH` by `BOARD_HEIGHT` 8-bit numbers, each number storing information about one cell. Cells are indexed by their row (`i`) and column (`j`), cell $(0, 0)$ being the top-left one. The least significant four bits out of these 8 represent the size of the city: 0 - if no city is in that cell, and a value between $0001_{(2)} = 1$ and $1111_{(2)} = 15$ if there is a city there with the respective population (that is why the maximal population of a city is 15, because it will fit in a 4-bit number). The most significant four bits from the number represent the directions in which there is a segment from that cell: 1 - if there is segment towards that direction, and 0 if not. The directions are established, from least significant to most significant bit among the four: right, down, left, up. Figure 9 illustrates the meaning of an element of the array pointed to by `data` (for one cell)

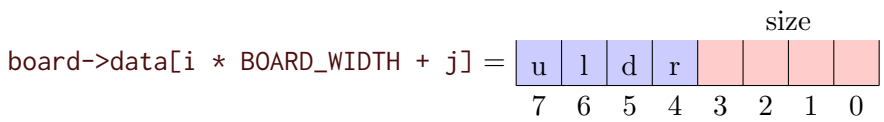
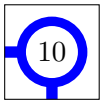


Figure 9: One cell information stored in an element of the array pointed to by `data`

Because this data is stored in an unidimensional array, then for cell (i, j) the information must be accessed as `board->data[i * BOARD_WIDTH + j]`. Figure 10 shows an example for a cell. Bits 0 to 3 represent the size of the city: $1010_{(2)} = 10$, bit 5 is 1 because there is rail segment downwards and bit 6 is 1 because there is rail segment leftwards.



a) the cell configuration

0	1	1	0	1	0	1	0
7	6	5	4	3	2	1	0

b) `board->data` for this cell

Figure 10: Example of `board->data`

There must be a constant tracking of the subnetworks that are formed into the board. The member `parents` of this structure keeps track of these subnetworks in the following manner: to each cell corresponds an element of the array pointed to by this pointer (just like for `data`). For each subnetwork that exists on the map, one cell is considered "parent", the representative of the subnetwork. Initially, there is no connection between any cell on the board, therefore each cell is its own subnetwork and its own "parent". This situation is depicted in Figure 11 for a small exemplification board.

		2			
				3	
1					
			9		

a) the actual board with the cities population

0	1	2	3	4	5
6	7	8	9	10	11
12	13	14	15	16	17
18	19	20	21	22	23

b) the value stored in `parents` for each cell

Figure 11: Initial state of the `parents` array

The initial attribution of the parents for each cell is performed in the `generate()` function as follows

```
int index = 0;
for(int i = 0; i < BOARD_HEIGHT; i++) {
    for(int j = 0; j < BOARD_WIDTH; j++) {
        board->data[i * BOARD_WIDTH + j] = 0;
        board->parents[i * BOARD_WIDTH + j] = index++;
    }
}
```

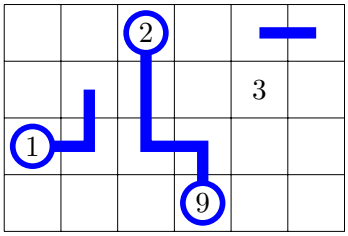
Initially, there is no segment therefore the most significant bits of the `data` array element for each cell are all 0. In this loop the whole `data` array element is initialized to 0 because the assignment of city populations is performed later.

Attention! Every cell is part of a subnetwork, even if it does not hold a city.

Now observe, after some connections were made, what changes have been produced in the `parents` array - see Figure 12.

It does not matter which cell is elected as representative. The algorithm just picks one of them but all that matters is that cells within the same subnetwork have the same representative.

The conditions to end a game can be rephrased in terms more appropriated to the way we store the data as



a) the actual board with the cities population

0	1	21	3	4	4
6	13	21	9	10	11
13	13	21	21	16	17
18	19	20	21	22	23

b) the value stored in `parents` for each cell

Figure 12: Some later state of the `parents` array

1. The game is **won** if the number of inhabited subnetworks (subnetworks that have at least one cell with population greater than 0 - there is city) is 1.
2. The game is **lost** if the current number of points is 0 and the condition above is not met.

Variable `int trees;` which is a local variable in the `play()` function represents the number of inhabited subnetworks.

Variable `int points;` stores the current number of points. Initially, it receives the value dictated by the `generate()` function which generated a board, adjusted to the difficulty the player has chosen.

3.1.2 The buffer

The data to be displayed on the LCD display is stored in an instance of the structure type defined in `play.h` as

```
typedef struct BUF {
  int width;
  int height;
  uint16_t *data;
} buf_t;
```

To each pixel on the display is associated a 16-bit value: least significant 5 bits for blue (values 0 to 31), next 6 bits for green (values 0 to 63) and the most significant 5 bits for red (values 0 to 31). Each pixel value is stored in an element of an array pointed to by `data`. The `width` and the `height` of the buffer equal the dimensions in pixels of the LCD display.

3.1.3 The selected and *under construction* cells

In order to keep track of these two important cells, we defined in `play.h` the structure

```
typedef struct CELL {
  int i, j;
} cell_t;
```

And the two cells of type `cell_t` are instatiated in the `game()` function, upon each game beginning

```
cell_t selected = {0, 0}, under_constr = {-1, 0};
```

If no cell is currently under construction, its `i` coordinate is set to `-1`, which is also the case at the beginning of the game.

3.2 Functions

Now we will explain how the most important functions in the game work. We will take them separately based on each source file that exists in the project: `main.c`, `play.c`, `events.c`, `rendering.c` and `generator.c`.

3.2.1 main.c

Here the function `main()` is implemented, which sets the connection with the board by calling `serialize_lock()`, and after the connection is confirmed, just the function `play()` is called.

3.2.2 play.c

play() In this file the function `play()` is called when the actual process of playing with the board starts. First, the preliminary functions to initialize the constants regarding input/output peripherals are called. Their implementation can be found in `rendering.c`. Also a buffer is instantiated.

```
init_rendering_constants();
init_reading_constants();
buf_t *buf = init_buffer();
```

Then the function `show_menu()`, implemented in `rendering.c` is called to display the main menu on the screen.

Then the algorithm runs through a loop which ensures the player can move through the menu options (the big loop in Figure 8). The variable `menu_row` takes values between 0 and 3 and stores the current active option on the menu. In each loop iteration, by the call of `listen_event()` it is checked whether a knob was rotated or pushed. If either the red or green knob was rotated, the current active menu option is changed: clockwise - downward (`listen_event()` returns `1` or `2`), counterclockwise - upward (`listen_event()` returns `3` or `4`). If any of the two knobs was pushed (`listen_event()` returns `5`), then the active menu option is selected. If the blue knob was pushed (`listen_event()` returns `6`), the whole process is aborted and the program run stops. Only if the option was to change the active option on the menu, the menu is displayed again

```
if(opt >= 1 && opt <= 4) {
    show_menu(menu_row, buf);
    put_buffer(buf);
}
```

otherwise it is only a waste of resources.

If the event returned by `listen_event()` is 5, meaning that the active option on the menu is selected, then we check

```
if(opt == 5) {
    if(menu_row == 3)
```

```
        opt = 6;
    else {
        win = game(menu_row + 1, buf);
        if(win != -1)
            write_final_message(buf, win);
        show_menu(menu_row, buf);
        put_buffer(buf);
    }
}
```

if the selected menu option was to exit (`menu_row` equal to `3`), we change the option to 6 which will stop iterating through the loop and the program will end. Otherwise, we call the function `game()`, which starts the actual game and returns the final result (which we store in the variable `win`). Its return value is `-1` if the game was aborted by pressing the blue knob, therefore, if that was not the case (so the game was either won or lost), we display the appropriate message on the screen by calling the function `write_final_message()`. After that message ends, the main menu is again displayed and we come back to the same loop.

Lastly, we test if the option was 6 (blue knob pushed), in such case we clear the displayed data, making the screen black.

When the main loop ends (the player wants to finish), the memory allocated for the buffer must be cleared.

game() This is probably the most important function in the code. When a new game is started, this function is called, and when the game ends this function terminates. At the beginning the data are initialized with the appropriate values and the function `generate()` is called to generate a board. The number of initial points is adjusted to the difficulty in the following way: the function `generate()` returns the number of initial points for the highest level of difficulty (for which it ensures that there is a possiblity to finish the game), but the actual number of initial points will be

```
int points = generate(&board, &trees) + 6 - 2 * difficulty;
```

Therefore, two extra points are awarded for each level of difficulty down.

Then, in our function the variable `event` holds the current input action taken by the player (same as `opt` in the `play()` function, just a different name). In each loop iteration (this loop representing the small loop in Figure 8), we call the function `listen_event()` to see if we receive any input data. `0` means that nothing happened. Otherwise, we check using a `switch(event)` structure. Events `1` to `4` correspond to knob rotation and therefore the selected cell will be moved if possible (if it is not already on the respective edge). Then, if the red or green knob was pressed (option `5`), there are more possibilities:

```
case 5:
    if(under_constr.i == selected.i && under_constr.j == selected.j) {
        under_constr.i = -1;
        break;
    }
    int link = are_neighbours(&under_constr, &selected);
    if(link != -1) {
```

```
        unify(&board, under_constr.i, under_constr.j, selected.i, selected.j, link, &points, &trees);
    }
    under_constr.i = selected.i;
    under_constr.j = selected.j;
```

if the selected cell coincides with the *under construction* cell, it just means that the player wants to deselect the *under construction* cell. Otherwise, we check whether the selected and *under construction* cell are neighbours by calling the approapriate function, which, if affirmative, returns a value between 0 and 3 corresponding with the direction in which the selected cell lays with respect to the *under construction* cell: 0 - right, 1 - down, 2 - left, 3 - up. If so, then a unification of subnetworks is performed by calling the function `unify()`. Then, as we said, in order to ease the rail construction, the selected cell now also becomes under construction.

After the `switch()` structure, we check whether the game is won (the number of populated subnetworks - variable `trees` equals 0). If so, for display reasons the number of points is reduced to 0 such that the LED strip will turn off. Then the buffer is redrawn according to the modified board, by calling the function `refresh_board()`.

Then, if one of the conditions to finish the game is met (either aborted: `event == 6` or won: `trees == 1` or lost: `points == 0`) we clear all the displayed data on the buffer and LEDs. Otherwise we put the buffer on the screen according to the board. After this, we return `0` if the game was won or `1` if lost.

In each loop iteration we make a pause of 1ms to not overwhelm the processor.

And if the loop ends it means that the event was to abort, therefore we return `-1`.

unify() this function unifies two subnetworks. Parameter `link` tells the direction of the segment. First we check whether between the two cells does not already exist a segment. If not, then one point is deduced for building the segment.

Then we draw the new segment by updating the `board->data` array according to the direction indicated by `link`. But for the selected cell we must draw the segment into the opposite direction than `link` (because `link` shows the way from *under construction* to *selected*). For this, we will use the great property of labeling the directions as we did, by which the opposite direction of `link` will always be `(link + 2) % 4`.

Later, the parent of each cell is taken from the `board->data` array: `parent0` for `under_constr` cell and `parent1` for `selected`. Also the sizes (number of cities) and populations (total number of populations of each city) in the two subnetworks are initialized with 0.

If the parents do not coincide (different parents), it means that we should actually perform a proper unification and we calculate the sizes and populations of the two subnetworks. If both of them are subnetworks which have at least one city (each size is greater than 0), then we test for each of them if their size is 1 (in such case it is a first time ever a city is connected and we must award the points for that) and also the total number of inhabited subnetworks decreases by 1, because we performed a unification between two of them.

In the end, we overwrite all the cells under the same parent (`parent0` for all of them).

3.2.3 events.c

This part of the code checks whether an input was sent from the player. In order to keep track of that, the global variables `prev_knob_red` and `prev_knob_green` store the previous rotary values given by the encoders of the knobs.

listen_event() In the body of this function the rotary encoded values of the knobs are read from the memory address `spiled_base + SPILED_REG_KNOBS_8BIT_o`, the least significant 8 bits read corresponding to the blue knob, the next 8 to the green one and the next 8 to the red knob.

Then various cases follow. In order to test whether the red knob moved, the condition is

```
if(knob_red != prev_knob_red && knob_red % 4 == 0)
```

because two things must happen: first of all, obviously the new value that is read must be different than the previous one, and also it is necessary to test if the current value is a multiple of 4, because the knobs are designed to rotate in small steps, not continuously, and the value that is read when a step is completed is multiple of 4. If we were not to test this, the function would return the result corresponding to rotation of the knob multiple times, even though we were rotating the knob only one step, and we would end up moving the selected cell for example multiple positions in only one step, which we obviously do not want.

Inside the `if` statement we have to check again the direction in which the knob was rotated, but we must also consider the fact that the encoded value is between 0 and 255 and it may overflow (because the knob can rotate indefinitely in any direction). Therefore we defined the constant `SAFETY_KNOB_DIFFERENCE` that represents a sefety margin, namely a maximal difference in knob position that can be achieved through rotation between two consecutive calls of this function if the encoded value *does not* overflow. Though, this margin is huge and this difference will never be achieved in reality. From this point, the logic is simple: if the current value is greater than the previous one, and also the difference between them is smaller than the margin, it means that the red knob was rotated clockwise and we return `2`, but if the difference is bigger it means that the encoded value has underflown and in fact the knob was rotated counterclockwise. And so on.

Then we check the same thing for the green knob.

In the end, we verify if the knobs were pushed. The 24th least significant bit in the value read from `spiled_base + SPILED_REG_KNOBS_8BIT_o` represents the push state of the blue knob (1 - pushed, 0 - not pushed). Than the bit next to it, more significant, corresponds to the green knob, the next one to the red knob. The variable `push_state` stores if one of the red or green knob was pushed from the last call of the function. We need this because while pushing a knob, the call of this function is surely performed many times because the loops where it is called from run fast, and the function would end up returning the value `5` multiple times during only one push, which we do not want. `push_state_blue` works exactly the same, but for the blue knob.

3.2.4 rendering.c

This section of the code deals with the output data, displayed on: LCD screen, LED strip and the RGB led. Writing data to the LCD screen is performed to the memory location

`parlcd_base`, which is mapped in the `init_rendering_constants()` function. Also the base memory address for the LEDs (`spiled_base`) is mapped in this function, where also the fonts for writing the city size, the main menu and the final message (after a game is won/lost) are chosen.

3.2.5 generator.c

In this file are defined 5 different boards, each of them along with a number of initial points. When generating a board is needed, the function `generate()` picks randomly one of the five boards and returns it.

4 Future improvements

To this date, there are only 5 predefined boards from which the algorithm randomly chooses one to play on. For the future, it would be great to have an algorithm implemented to generate different boards all the time. The trickier part is for that algorithm to compute the necessary number of initial points, such as a solution is possible but also the game is challenging.

References

[1] Pavel Píša - *Computer Architectures - Lecture 8*, lecture notes, Czech Technical University in Prague, 2025